



INSTITUTO TECNOLÓGICO DE BUENOS AIRES

72.41 Base de Datos II

TRABAJO PRÁCTICO OBLIGATORIO

Grupo 14

Integrantes del grupo:

64237	Balboa Koolen, Tomas Rafael
64233	Stupnik, Luciano
64756	Esquivel, Tomas Jeronimo

Introducción:	3
Tecnologías Utilizadas:	3
Justificación de la Persistencia Políglota:	3
MongoDB: Persistencia Documental	4
Redis: Caché de Consultas	5
4. Esquema de base de datos	6
Esquema Lógico (MongoDB)	7
Desnormalización	10
Índices	11
Registros Agregados	11
Esquema Lógico (Redis)	12
Esquema Físico (Docker Compose)	13
5. Comandos	14
6. Conclusiones, limitaciones y posibles extensiones	22

Introducción:

Este informe detalla el diseño y la implementación del backoffice de VetSalud S.A., una red de clínicas veterinarias con presencia en varias provincias argentinas, según lo solicitado en el Trabajo Práctico Obligatorio de Base de Datos II. El objetivo principal fue desarrollar una solución robusta capaz de gestionar la información de los pacientes de las veterinarias (animales), propietarios, veterinarios, consultas médicas, vacunaciones, cirugías y productos del stock farmacéutico.

Es requisito del trabajo práctico utilizar una capa de persistencia políglota que combine dos motores NoSQL. Nuestra solución utiliza MongoDB como base de datos principal, siendo esta misma una base de datos documental, y a Redis como una capa extra de caché para optimizar las consultas de lectura.

A continuación se presentará la justificación de la selección de las tecnologías ya mencionadas, los esquemas lógicos y físicos utilizados y los comandos necesarios para hacer las consultas al backoffice de VetSalud S.A.

Tecnologías Utilizadas:

- **Base de Datos Principal:** MongoDB
- **Base de Datos de Caché:** Redis
- **Plataforma / Lenguaje:** Python 3.11 con FastAPI
- **Drivers de bases de datos:** PyMongo (driver oficial de MongoDB) y redis-py (cliente oficial de Redis)
- **Entorno de Ejecución:** Docker (utilizando Docker Compose)

Justificación de la Persistencia Políglota:

Para el desarrollo de VetSalud se adoptó una arquitectura de persistencia políglota basada en dos motores NoSQL de paradigmas diferentes: MongoDB, como base de datos documental y fuente de verdad del sistema, y Redis, como almacén clave-valor en memoria utilizado para implementar una capa de caché. En otras palabras, MongoDB conserva la verdad del negocio y Redis acelera su consulta.

Aunque el trabajo de por sí ya solicitaba el uso de dos motores de bases de datos NOSQL de paradigmas distintos, el foco principal estuvo puesto en asignar a cada motor el trabajo para el cual resultaba más adecuado. MongoDB prioriza la durabilidad, la organización documental y las consultas complejas. Por otro lado, Redis prioriza la velocidad de acceso a resultados consultados con frecuencia

MongoDB: Persistencia Documental

MongoDB almacena de forma permanente todas las entidades del dominio:

1. Propietarios
2. Pacientes
3. Veterinarios
4. Consultas médicas
5. Vacunaciones
6. Cirugías
7. Stock farmacéutico.

La elección de MongoDB por sobre otros motores, se relaciona con las siguientes necesidades:

- **Representación del dominio:** Las entidades de VetSalud poseen múltiples atributos relacionados. Por ejemplo, una consulta incluye el paciente, el veterinario, la fecha, el motivo, el diagnóstico, el costo y el estado. El modelo documental permite representar cada atención mediante un único documento similar a los objetos utilizados por la aplicación.
- **Consultas Agregadas:** Varias consignas requieren combinar, agrupar u ordenar información. Por ejemplo, el historial de un paciente une consultas y vacunaciones; el ranking de diagnósticos agrupa las consultas por diagnóstico; y la vista de ingresos mensuales suma los costos por veterinario. Estas operaciones pueden resolverse mediante el Aggregation Framework, utilizando etapas como \$lookup, \$group, \$unionWith y \$sort.
- **Relaciones entre entidades:** Aunque MongoDB no utiliza relaciones rígidas como una base relacional, permite vincular colecciones mediante identificadores. Por ejemplo, cada paciente guarda el identificador de su propietario y cada consulta referencia al paciente y al veterinario. Esto permite resolver consultas como “pacientes activos con todos los datos de su propietario” mediante \$lookup.
- **Indexación:** El sistema realiza búsquedas frecuentes por identificadores y fechas. Por ejemplo, el historial filtra por id_paciente, las consultas de los últimos 60 días utilizan fecha, las vacunas vencidas utilizan proxima_dosis y el stock bajo filtra por unidades. Por ello se crearon índices sobre estos campos para reducir la cantidad de documentos examinados.
- **Durabilidad:** MongoDB conserva toda la información de negocio, incluido el stock farmacéutico, evitando que un dato crítico dependa de una caché en memoria.

- **Flexibilidad frente a evolución del sistema:** Aunque actualmente los documentos de cada colección mantienen una estructura uniforme, MongoDB permitiría incorporar nuevos datos clínicos, como medicamentos indicados, observaciones o estudios, sin exigir que todos los registros históricos sean modificados inmediatamente. Esta característica facilita futuras ampliaciones del sistema, aunque no constituye el motivo principal de su elección en la implementación actual.

Por último, en algunas consultas se utiliza una desnormalización controlada de datos del veterinario y del propietario. Esto disminuye la cantidad de uniones necesarias y conserva los datos correspondientes al momento de la atención.

Redis: Caché de Consultas

Redis se utiliza como caché distribuida para las consultas de lectura de la API. Su modelo clave-valor y su funcionamiento en memoria permiten recuperar resultados con una latencia muy baja. Esta decisión resulta adecuada porque varias consultas requieren recorrer, relacionar, agrupar u ordenar documentos de MongoDB. Consecuentemente, si distintos usuarios solicitan repetidamente la misma información, volver a ejecutar estas operaciones produce trabajo innecesario sobre la base principal.

Se utilizó Redis implementando el patrón cache-aside:

1. La aplicación primero busca el resultado en Redis.
2. Si existe, lo devuelve directamente.
3. Si no existe, ejecuta la consulta en MongoDB.
4. Guarda temporalmente el resultado en Redis.
5. Devuelve la respuesta obtenida.

Este mecanismo resulta especialmente útil para consultas que realizan agrupaciones, ordenamientos o relaciones entre colecciones, como el ranking de diagnósticos, los ingresos mensuales y los historiales de pacientes. Al reutilizar resultados recientes, se reduce la cantidad de operaciones repetidas sobre MongoDB.

Lógicamente, se tuvo que tener en cuenta que si una operación modifica datos, se invalidan las claves relacionadas (con el objetivo de mantener la coherencia). Por ejemplo, registrar una consulta elimina de la caché el historial del paciente, el ranking de diagnósticos y los ingresos por veterinario. La siguiente lectura vuelve a obtener la información actualizada desde MongoDB.

Además, cada entrada posee un **TTL**: una hora para consultas normales y doce horas para agregaciones relativamente estables. El TTL funciona como un límite adicional para evitar que un resultado permanezca indefinidamente almacenado.

Esta utilización de Redis aporta los siguientes beneficios:

- **Menor tiempo de respuesta:** Como ya se mencionó, Redis trabaja principalmente en memoria, por lo que recuperar un resultado almacenado resulta más rápido que volver a ejecutar un pipeline de agregación en MongoDB.
- **Reducción de carga sobre MongoDB:** Consultas repetidas, como el ranking de diagnósticos, los ingresos mensuales o los pacientes de una sucursal, pueden resolverse desde Redis sin volver a recorrer las colecciones.
- **Mejor aprovechamiento según el tipo de consulta:** Los datos se mantienen en MongoDB, mientras que Redis conserva únicamente resultados ya calculados. De esta forma se utilizan las capacidades de agregación de MongoDB y la rapidez de lectura de Redis.

Es importante reforzar la idea de que Redis no contiene la información definitiva. Si deja de estar disponible, el sistema continúa consultando MongoDB, aunque con una posible disminución del rendimiento. Por lo tanto, Redis actúa como acelerador y MongoDB permanece como única fuente de verdad.

4. Esquema de base de datos

Antes de detallar el esquema lógico y físico, es necesario aclarar ciertas decisiones que se tomaron sobre el dominio y el dataset, ya que condicionan tanto la estructura de las colecciones como el comportamiento de las consultas.

En primer lugar, todas las entidades se identifican mediante una clave natural provista por el negocio (*id_propietario*, *id_paciente*, *id_vet*, *id_consulta*, *id_vacuna*, *id_cirugia*, *id_producto*) y no mediante el *_id* autogenerado por MongoDB. Estas claves son las que aparecen en los datasets de la cátedra y las que utiliza la API en sus rutas, por lo que se las declaró como índices únicos para garantizar su unicidad y acelerar las búsquedas por identificador.

En segundo lugar, las bajas de propietarios se resuelven como una baja lógica y no como una eliminación física. Dar de baja un propietario consiste en marcar su campo activo en *false*, de modo que las consultas que piden explícitamente entidades activas (por ejemplo, pacientes activos con sus datos de propietario) lo excluyan, mientras que el resto de las consultas y, sobre todo, el historial clínico de sus pacientes, conservan la información. El mismo criterio de activo se aplica a pacientes y veterinarios.

En tercer lugar, las entidades del dominio se dividen conceptualmente en entidades maestras (propietarios, veterinarios, pacientes y stock), que representan un estado actual, y

entidades de evento (consultas, vacunaciones y cirugías), que representan un hecho ocurrido en una fecha determinada y que no debe reescribirse retroactivamente. Esta distinción es la que justifica la estrategia de desnormalización descrita más adelante.

Por ultimo, salvo que una consulta lo solicite de forma específica, no se filtran los resultados según el estado de actividad de la entidad ni se aplican reglas de negocio adicionales (por ejemplo, no se asume que sólo deban existir consultas en un estado determinado). Las validaciones se limitan a lo estrictamente pedido: que el paciente y el veterinario existan al registrar una consulta, y que haya unidades suficientes al decrementar stock.

Esquema Lógico (MongoDB)

La base de datos en MongoDB consta de siete colecciones, una por cada entidad del dominio. Las relaciones entre ellas no se modelan mediante claves foráneas a nivel motor, sino mediante la referencia de identificadores naturales, resueltas en tiempo de consulta a través del Aggregation Framework (*\$lookup*, *\$group*, *\$unionWith*, *\$sort*).

A diferencia de otros dominios, no se embeben las entidades de evento dentro de las maestras: embeber el conjunto de consultas, vacunaciones y cirugías dentro de cada paciente generaría un arreglo no acotado que crece sin límite a lo largo de la vida del animal, penalizando toda lectura del paciente. Por ello, cada evento vive en su propia colección y referencia al paciente y al veterinario por identificador.

A continuación se describen las colecciones y sus campos principales:

- **propietarios:**

Campo	Tipo
id_propietario	str
nombre, apellido	str
dni	str
email, telefono	str
ciudad, provincia	str
activo	bool

- **veterinarios:**

Campo	Tipo
id_vet	str
nombre, apellido	str
matricula	str
especialidad	str
sucursal	str
activo	bool

- **pacientes:**

Campo	Tipo
id_paciente	str
nombre, especie, raza	str
fecha_nac	datetime
id_propietario	str (FK)
activo	bool

- **consultas:**

Campo	Tipo
id_consulta	str
id_paciente	str (FK)
id_vet	str (FK)
fecha	datetime
motivo, diagnostico	str
consto	number
estado	str

vet_nombre, vet_apellido	str
vet_especialidad, vet_sucursal	str
id_propietario	str (FK)

- **vacunaciones:** Incluye snapshot del veterinario.

Campo	Tipo
id_vacuna	str
id_paciente	str (FK)
id_vet	str (FK)
fecha_aplicacion	datetime
nombre_vacuna	str
proxima_dosis	datetime
vet_nombre, vet_apellido, vet_especialidad, vet_sucursal	str

- **cirugias:**

Campo	Tipo
id_cirugia	str
id_paciente	str (FK)
id_vet	str (FK)
fecha	datetime
tipo	str
resultado	str
costo	number

- **stock:** inventario farmacéutico (entidad aislada)

Campo	Tipo
id_producto	str

nombre, categoria	str
unidades	int
precio_unit	number
vencimiento	datetime
proveedor	str

La colección stock no participa de ninguna relación: ningún id_producto es referenciado por otra colección. Se trata de una entidad aislada (un solo documento), lo que la convierte en el caso ideal para validar la atomicidad del decremento de unidades, ya que MongoDB garantiza que las operaciones sobre un sólo documento son atómicas.

Desnormalización

El dominio presenta dos cadenas de doble salto (lookup) que se resuelven con una desnormalización dirigida, de modo que las consultas más frecuentes eviten uniones encadenadas:

- **Snapshot del veterinario en cada evento:** las consultas, vacunaciones y cirugías guardan una copia de vet_nombre, vet_apellido, vet_especialidad y vet_sucursal correspondiente al momento de la atención. Esto permite, por ejemplo, agrupar ingresos por veterinario o filtrar pacientes por sucursal sin necesidad de unir contra veterinarios, y además conserva el dato histórico: si un profesional cambia de sucursal, los eventos pasados mantienen la sucursal en la que efectivamente se realizaron.
- **Dueño desnormalizado en cada consulta:** cada consulta almacena el id_propietario del paciente atendido, evitando el doble salto consultas → pacientes → propietarios en las consultas que relacionan atenciones con dueños (por ejemplo, propietarios sin consultas en el último año).

En vez de embeber documentos enteros, se embeben sets acotados de atributos cortos y estables. De esta manera, nos aseguramos un volumen mínimo por copia.

Además, las dos cadenas son 1:N (1 veterinario : N eventos y 1 dueño : N consultas) y el dato copiado vive en el lado N. Esto podría ser un problema por la anomalía de actualización, sin embargo nuestro diseño neutraliza esa anomalía:

- El evento es una copia histórica, que no debe ser actualizada ya que está congelada al momento del evento
- Para el dueño se copia sólo la FK, por lo que un cambio en otro atributo del dueño no invalida al atributo embebido. También, el dueño de un paciente en una consulta

queda desactualizado únicamente si el paciente cambia de propietario, suceso extremadamente extraño.

Índices

Sobre la base de los patrones de consulta del sistema, se definieron los siguientes índices:

- **propietarios:** id_propietario (único)
- **veterinarios:** id_vet (único)
- **pacientes:** id_paciente (único), id_propietario
- **consultas:** id_consulta (único), id_paciente, id_vet, fecha, vet_sucursal, id_propietario
- **vacunaciones:** id_vacuna (único), id_paciente, proxima_dosis
- **cirugías:** id_cirugia (único)
- **stock:** id_producto (único), unidades, vencimiento

Los índices únicos sobre las claves naturales garantizan la unicidad y aceleran las búsquedas por identificador. Los índices sobre fecha, proxima_dosis y unidades sostienen las consultas sensibles al tiempo (actividad reciente de veterinarios, vacunas vencidas) y los filtros por stock bajo. Por último, los índices sobre vet_sucursal e id_propietario en consultas existen específicamente para aprovechar las desnormalizaciones descritas, acelerando el filtrado y la agrupación de las consultas que las utilizan.

Registros Agregados

El enunciado exige incorporar al menos diez registros propios por colección, además de los provistos en los datasets de la cátedra. El poblamiento se realiza en dos etapas: primero un ETL (Extract, Transform, Load) que carga y limpia los CSV de la cátedra (conversión de fechas a datetime, normalización de montos y categorías, validación de integridad referencial e índices) y, a continuación, agregamos los registros adicionales propios.

Ahora bien, estos registros adicionales no se generaron de forma arbitraria. Dado que varias de las consultas son sensibles a la fecha o dependen de condiciones particulares del dataset, se adoptó un criterio de diseño "de atrás hacia adelante": partiendo de cada una de las quince consultas y servicios, se determinó qué condiciones debían cumplir los datos para que esa consulta devolviera un resultado significativo y no vacío, y recién entonces se construyeron los registros que las satisfacen. Para que las consultas dependientes del tiempo sean deterministas, todo el conjunto se ancló a una fecha de referencia (2026-06-01).

Bajo este criterio, los registros adicionales se diseñaron, entre otros objetivos, para garantizar que:

- existan **consultas dentro de los últimos 60 días** y del mes en curso distribuidas entre varios veterinarios, de modo que las consultas de actividad reciente por veterinario y de ingresos mensuales muestren acumulación real;
- haya vacunaciones con la **próxima dosis ya vencida** respecto de la fecha de referencia, junto con otras vigentes, para que la consulta de vacunas vencidas distinga ambos casos;
- existan **propietarios sin consultas** y al menos un propietario cuya última consulta es anterior al último año, para poblar la consulta de propietarios inactivos;
- los diagnósticos se repitan (Gastritis, Sano, Dermatitis atópica, etc.) para que el Top 5 de diagnósticos más frecuentes sea no trivial;
- haya consultas de motivo "Control" con costo inferior al umbral y varias consultas en estado "Seguimiento", para las consultas de controles de **bajo costo** y de atenciones abiertas;
- el stock incluya productos con **menos de cincuenta unidades** junto a otros por encima de ese umbral, para la consulta de stock bajo;
- haya propietarios con más de un paciente y veterinarios asignados a una misma sucursal, para las consultas de **propietarios multi-paciente** y de pacientes por sucursal;
- la colección de cirugías, que no posee datos en los CSV de la cátedra, se pueble por completo de forma coherente con los pacientes y veterinarios existentes.

De este modo, cada consulta cuenta con datos representativos que ejercitan su lógica, y la integridad referencial del conjunto queda garantizada: la carga falla de forma ruidosa si algún evento referencia un paciente o veterinario inexistente, por lo que no quedan referencias colgadas.

Esquema Lógico (Redis)

El esquema de Redis es un almacén clave-valor simple, utilizado exclusivamente como capa de caché de lectura. Las claves se construyen de forma estandarizada mediante una función central (`cache_key`), que las prefija con `cache:` y las compone a partir del identificador de la consulta y de sus parámetros, de modo que cada variante parametrizada de una misma consulta posea su propia entrada. Los valores son el resultado de la consulta a MongoDB, serializado como una cadena JSON (con las fechas normalizadas a formato ISO para que un acierto y un fallo de caché devuelvan exactamente el mismo formato).

Ejemplos de claves:

- **cache:q02:** consultas en estado Seguimiento.
- **cache:q05:** veterinarios activos y sus consultas de los últimos 60 días.
- **cache:q07:5:** Top 5 diagnósticos más frecuentes (el 5 es el parámetro límite).
- **cache:q09:5000:** consultas de tipo Control con costo menor al umbral indicado.

La lógica de la aplicación gestiona la lectura y escritura de la caché desde la capa de servicios mediante el patrón cache-aside: cada lectura se envuelve en `get_or_set_cache`, que sirve el valor desde Redis si existe o, en caso contrario, ejecuta la consulta sobre MongoDB, almacena el resultado y lo devuelve. Cada escritura (alta, modificación o baja de propietarios, registro de consulta o decremento de stock) dispara la invalidación de las claves de las consultas afectadas mediante `invalidate`, de modo que la próxima lectura recurra a MongoDB y vuelva a poblar la caché con el estado fresco.

Se definieron dos tiempos de vida (TTL): 1 hora para las consultas normales y 12 horas para los agregados casi estáticos y costosos de recalcular (por ejemplo, el Top de diagnósticos). Dado que la invalidación explícita garantiza que nunca se sirvan datos inconsistentes con la fuente de verdad, el TTL actúa únicamente como red de seguridad: si al querer invalidar un dato, REDIS está caído, al volverse a levantar mantiene el dato que no pudo ser invalidado. Esto genera una inconsistencia que únicamente se resuelve gracias al TTL.

Esquema Físico (Docker Compose)

El entorno de despliegue se gestiona íntegramente a través de `docker-compose.yml`, que define y orquesta los servicios necesarios:

- **mongo:** contenedor oficial de MongoDB (imagen `mongo:7`), con su estado persistido en un volumen nombrado (`mongo_data`). Expone el puerto 27017 y posee un healthcheck que verifica su disponibilidad antes de que arranque la API.
- **redis:** contenedor oficial de Redis (imagen `redis:7`), con persistencia AOF y su propio volumen (`redis_data`). Expone el puerto 6379 y cuenta también con un healthcheck.
- **app:** contenedor de la aplicación, construido a partir del Dockerfile local, que ejecuta la API REST (FastAPI sobre Uvicorn) en el puerto 8000. Arranca recién cuando MongoDB y Redis están healthy y no carga datos por sí mismo: utiliza lo que haya en los volúmenes.
- **etl:** servicio a demanda (protegido por un profile) que ejecuta la carga limpia y la siembra de datos. Se invoca manualmente cuando se desea poblar o resetear el dataset.
- **tests:** servicio a demanda que ejecuta la suite de pruebas sobre una base de datos aislada (`vetsalud_test`), sin afectar los datos de trabajo.

Todos los contenedores se comunican dentro de la red interna de Docker, lo que permite que el servicio `app` se conecte a `mongo` y a `redis` utilizando sus nombres de servicio como nombres de host. La persistencia en volúmenes nombrados garantiza que el estado de ambas bases sobreviva a la detención de los contenedores (`docker compose down`) y sólo se pierda al eliminarlos explícitamente (`docker compose down -v`). Esta configuración asegura un entorno de despliegue consistente, aislado y reproducible.

5. Comandos

A continuación, se presentan los endpoints de la API REST y los comandos necesarios para interactuar con la misma y utilizar los servicios solicitados.

A lo largo de esta sección, utilizaremos cURL para mostrar los resultados, pero se pueden utilizar otras herramientas. Asumiremos que la API corre en localhost:8000

Para poder visualizar fácilmente las diferentes funcionalidades de la API, se puede abrir <http://localhost:8000/docs> desde un navegador con la API corriendo.

1. Pacientes activos con todos sus datos de propietario.

1. **Endpoint:** GET /pacientes/activos
2. **Ejecucion:**

```
curl -X 'GET' 'http://localhost:8000/pacientes/activos' \
-H 'accept: application/json'
```

3. **Resultado:**

```
[
  {
    "id_paciente": "P001",
    "nombre": "Firulais",
    "especie": "Perro",
    "raza": "Labrador",
    "fecha_nac": "2019-04-12T00:00:00",
    "propietario": {
      "id_propietario": "C001",
      "nombre": "Ana",
      "apellido": "Rodríguez",
      "dni": "30456789",
      "email": "ana@gmail.com",
      "telefono": "1145001234",
      "ciudad": "Buenos Aires",
      "provincia": "Buenos Aires"
    },
    ... (muchos registros de pacientes)
  },
  ... (muchos registros de pacientes)
]
```

2. Consultas médicas abiertas (estado 'Seguimiento') con veterinario asignado y costo.

1. **Endpoint:** GET /consultas/seguimiento
2. **Ejecucion:**

```
curl -X 'GET' 'http://localhost:8000/consultas/seguimiento' \
-H 'accept: application/json'
```

3. **Resultado:**

```
[
  {
    "id_consulta": "CON004",
    "id_paciente": "P004",
    "fecha": "2025-11-01T00:00:00",
    "motivo": "Pérdida de apetito",
    "diagnostico": "Estrés ambiental",
    "costo": 3800,
    "veterinario": {
      "id_vet": "V001",
      "nombre": "Gabriela",
      "apellido": "Fontana",
      "especialidad": "Clínica General",
      "sucursal": "Palermo"
    }
  },
  (... muchos elementos)
]
```

3. Historial completo de un paciente: consultas y vacunaciones ordenadas por fecha.

1. **Endpoint:** GET /pacientes/{id-paciente}/historial
2. **Ejecucion (con id P001):**

```
curl -X 'GET' \
'http://localhost:8000/pacientes/P001/historial-completo' \
-H 'accept: application/json'
```

3. **Resultado:**

```
[
  {
    "tipo": "Consulta",
    "fecha": "2025-10-05T00:00:00",
    "detalle": "Control anual",
    "diagnostico": "Sano",
    "id_vet": "V001"
  },
  {
    "tipo": "Vacunación",
    "fecha": "2025-10-05T00:00:00",
    "detalle": "Antirrábica",
    "diagnostico": null,
    "id_vet": "V001"
  },
  (... mas registros)
]
```

4. Propietarios con más de un paciente registrado.

1. **Endopint:** GET /propietarios/multi-pacientes
2. **Ejecucion (con id P001):**

```
curl -X 'GET' \
  'http://localhost:8000/propietarios/multi-paciente' \
  -H 'accept: application/json'
```

3. **Resultado:**

```
[
  {
    "cantidad_pacientes": 2,
    "pacientes": [
      {
        "id_paciente": "P001",
        "nombre": "Firulais",
        "especie": "Perro",
        "raza": "Labrador",
        "fecha_nac": "2019-04-12T00:00:00",
        "activo": true
      },
      {
        "id_paciente": "P003",
        "nombre": "Coco",
        "especie": "Perro",
        "raza": "Beagle",
        "fecha_nac": "2018-11-20T00:00:00",
        "activo": false
      }
    ],
    "id_propietario": "C001",
    "nombre": "Ana",
    "apellido": "Rodríguez",
    "dni": "30456789",
    "email": "ana@gmail.com",
    "telefono": "1145001234",
    "ciudad": "Buenos Aires",
    "provincia": "Buenos Aires"
  },
  (... mas registros)
]
```


5. Veterinarios activos y cantidad de consultas realizadas en los últimos 60 días.

1. **Endpoint:** GET /consultas/vets-activos
2. **Ejecucion:**

```
curl -X 'GET' \
  'http://localhost:8000/consultas/vets-activos' \
  -H 'accept: application/json'
```

3. **Resultado:**

```
{
  "id_vet": "V006",
  "nombre": "Diego",
  "apellido": "Morales",
  "especialidad": "Clínica General",
  "sucursal": "Núñez",
  "consultas_60d": 6
},
  (... más registros)
}
```

6. Pacientes con vacunas vencidas (próxima dosis anterior a hoy).

1. **Endpoint:** GET /pacientes/vacunas-vencidas
2. **Ejecucion:**

```
curl -X 'GET' \
  'http://localhost:8000/pacientes/vacunas-vencidas' \
  -H 'accept: application/json'
```

3. **Resultado:**

```
[
  {
    "vacunas_vencidas": [
      {
        "id_vacuna": "VAC019",
        "nombre_vacuna": "Sextuple",
        "fecha_aplicacion": "2025-03-12T00:00:00",
        "proxima_dosis": "2026-03-12T00:00:00"
      },
      {
        "id_vacuna": "VAC010",
        "nombre_vacuna": "Antirrábica",
        "fecha_aplicacion": "2025-05-20T00:00:00",
        "proxima_dosis": "2026-05-20T00:00:00"
      }
    ],
    "id_paciente": "P002",
    "nombre": "Michi",
    "especie": "Gato",
    "raza": "Siamés",
    "fecha_nac": "2021-07-03T00:00:00",
    "id_propietario": "C002",
    "activo": true
  },
  (... mas datos)
]
```

7. Top 5 diagnósticos más frecuentes.

1. **Endpoint:** GET /consultas/top-diagnosticos

2. **Ejecucion:**

```
curl -X 'GET' \
  'http://localhost:8000/consultas/top-diagnosticos?limite=5' \
  -H 'accept: application/json'
```

3. **Resultado:**

```
[
  {
    "frecuencia": 8,
    "diagnostico": "Sano"
  },
  (... más registros)
  {
    "frecuencia": 2,
    "diagnostico": "Parásitos intestinales"
  }
]
```

8. Stock de productos con menos de 50 unidades y su proveedor.

1. **Endpoint:** GET /stock/bajo

2. **Ejecucion:**

```
curl -X 'GET' \
  'http://localhost:8000/stock/bajo?umbral=50' \
  -H 'accept: application/json'
```

3. **Resultado:**

```
[
  {
    "id_producto": "PRD016",
    "nombre": "Furosemida 40mg",
    "categoria": "Diurético",
    "unidades": 20,
    "precio_unit": 1450,
    "proveedor": "VetFarma SA"
  },
  (... más registros)
  {
    "id_producto": "PRD009",
    "nombre": "Tramadol 50mg",
    "categoria": "Analgésico",
    "unidades": 45,
    "precio_unit": 2100,
    "proveedor": "MediAnimal"
  }
]
```

9. Consultas de tipo 'Control' con costo menor a \$5.000.

1. **Endopint:** GET /consultas/control-bajo-costo
2. **Ejecucion:**

```
curl -X 'GET' \  
'http://localhost:8000/consultas/control-bajo-costo?umbral=5000' \  
-H 'accept: application/json'
```

3. **Resultado:**

```
[  
  {  
    "id_consulta": "CON007",  
    "id_paciente": "P001",  
    "id_vet": "V001",  
    "fecha": "2025-12-02T00:00:00",  
    "motivo": "Control post-vacuna",  
    "diagnostico": "Sin novedades",  
    "costo": 2000  
  },  
  (... mas registros)  
  {  
    "id_consulta": "CON001",  
    "id_paciente": "P001",  
    "id_vet": "V001",  
    "fecha": "2025-10-05T00:00:00",  
    "motivo": "Control anual",  
    "diagnostico": "Sano",  
    "costo": 4500  
  }  
]
```

10. Todos los pacientes de una sucursal determinada (a través del veterinario).

1. **Endopint:** GET /pacientes/por-sucursal
2. **Ejecucion (con sucursal Palermo):**

```
curl -X 'GET' \  
'http://localhost:8000/pacientes/por-sucursal?sucursal=Palermo' \  
-H 'accept: application/json'
```

3. **Resultado:**

```
[  
  {  
    "veterinarios": [  
      {  
        "id_vet": "V001",  
        "nombre": "Gabriela",  
        "apellido": "Fontana",  
        "especialidad": "Clínica General",  
        "sucursal": "Palermo"  
      }  
    ],  
    "id_paciente": "P001",  
    "nombre": "Firulais",  
    "especie": "Perro",  
    "raza": "Labrador",  
    "activo": true,  
    "id_propietario": "C001",  
  },  
  (... mas registros)  
]
```

11. Vista agregada: ingresos totales por veterinario en el mes actual.

1. **Endopint:** GET /consultas/ingresos-por-vet

2. **Ejecucion:**

```
curl -X 'GET' \
  'http://localhost:8000/consultas/ingresos-por-vet' \
  -H 'accept: application/json'
```

3. **Resultado:**

```
[
  {
    "ingresos": 18800,
    "id_vet": "V003",
    "nombre": "Cecilia",
    "apellido": "Rios",
    "especialidad": "Dermatologia",
    "sucursal": "Palermo"
  },
  (... más registros)
]
```

12. Propietarios sin consultas registradas en el último año.

1. **Endopint:** GET /propietarios/sin-consultas

2. **Ejecucion:**

```
curl -X 'GET' \
  'http://localhost:8000/propietarios/sin-consultas' \
  -H 'accept: application/json'
```

3. **Resultado:**

```
[
  {
    "id_propietario": "C006",
    "nombre": "Tomás",
    "apellido": "Juárez",
    "dni": "42345678",
    "email": "tomas@gmail.com",
    "telefono": "1190006789",
    "ciudad": "Salta",
    "provincia": "Salta"
  },
  (... mas registros)
  {
    "id_propietario": "C015",
    "nombre": "Mariana",
    "apellido": "Acosta",
    "dni": "41222333",
    "email": "mariana@gmail.com",
    "telefono": "1145990099",
    "ciudad": "Neuquén",
    "provincia": "Neuquén"
  }
]
```

13. ABM completo de propietarios: alta, modificación de datos, baja lógica.

ALTA DE PROPIETARIOS

1. **Endpoint:** POST /propietarios/
2. **Ejecucion:**

```
curl -X 'POST' \
  'http://localhost:8000/propietarios' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{"id_propietario": "C900",
"nombre": "Ada",
"apellido": "Lovelace",
"dni": "12345678",
"email": "ada@vetsalud.com",
"telefono": "1144440000",
"ciudad": "CABA",
"provincia": "Buenos Aires",
"activo": true}'
```

3. Resultado:

```
{
  "activo": true,
  "id_propietario": "C900",
  "nombre": "Ada",
  "apellido": "Lovelace",
  "dni": "12345678",
  "email": "ada@vetsalud.com",
  "telefono": "1144440000",
  "ciudad": "CABA",
  "provincia": "Buenos Aires"
}
```

MODIFICACION DE PROPIETARIOS

1. **Endpoint:** PUT /propietarios/{id_propietario}
2. **Ejecucion:**

```
curl -X 'PUT' \
  'http://localhost:8000/propietarios/C900' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{"id_propietario": "C900",
"nombre": "Ya no es ADA",
"email": "yanoesada@vetsalud.com"}'
```

Se modifican los campos que se envíen

3. Resultado:

```
{
  "activo": true,
  "id_propietario": "C900",
  "nombre": "Ya no es ADA",
  "apellido": "Lovelace",
  "dni": "",
  "email": "yanoesada@vetsalud.com",
  "telefono": "1144440000",
  "ciudad": "CABA",
  "provincia": "Buenos Aires"
}
```

BAJA LOGICA DE PROPIETARIOS

1. **Endpoint:** DELETE /propietarios/{id_propietario}
2. **Ejecucion:**

```
curl -X 'DELETE' \
  'http://localhost:8000/propietarios/C900' \
  -H 'accept: application/json'
```

3. Resultado:

```
{
  "activo": false,
  "id_propietario": "C900",
  "nombre": "Ya no es ADA",
  "apellido": "Lovelace",
  "dni": "",
  "email": "yanoesada@vetsalud.com",
  "telefono": "1144440000",
  "ciudad": "CABA",
  "provincia": "Buenos Aires"
}
```

14. Registro de nueva consulta médica con validación de paciente y veterinario existentes.

1. **Endpoint:** POST /consultas

CASO 1: Veterinario y paciente correctos

1. **Ejecucion:**

```
curl -X 'POST' \
  'http://localhost:8000/consultas' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{"id_paciente": "P001",
"vet": "V001",
"motivo": "Control anual",
"diagnostico": "Sano",
"costo": 3500,
"estado": "Cerrada"}'
```

2. Resultado:

```
Response CODE: 200
{
  "id_consulta": "CON032",
  "id_paciente": "P001",
  "id_vet": "V001",
  "fecha": "2026-06-10T15:49:12.293878",
  "motivo": "Control anual",
  "diagnostico": "Sano",
  "costo": 3500,
  "estado": "Cerrada",
  "vet_nombre": "Gabriela",
  "vet_apellido": "Fontana",
  "vet_especialidad": "Clínica General",
  "vet_sucursal": "Palermo",
  "id_propietario": "C001"
}
```

CASO 2: Veterinario o paciente incorrectos

1. **Ejecucion:**

```
curl -X 'POST' \
  'http://localhost:8000/consultas' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{"id_paciente": "esteidnoexiste",
"vet": "V001",
"motivo": "Control anual",
"diagnostico": "Sano",
"costo": 3500,
"estado": "Cerrada"}'
```

2. Resultado:

```
Response CODE: 404
{
  "detail": "El paciente esteidnoexiste no existe"
}

Si ponemos un veterinario que no existe:

{
  "detail": "El veterinario ? no existe"
}
```

15. Actualización masiva del stock: decrementar unidades de un producto tras una consulta.

1. Endopint: POST /stock/{id_producto}/decrementar

CASO 1: Hay suficientes productos para decrementar

1. Ejecucion (producto PRD001):

```
curl -X 'POST' \
'http://localhost:8000/stock/PRD001/decrementar' \
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
  "cantidad": 3
}'
```

2. Resultado:

Response CODE: 200

```
{
  "id_producto": "PRD001",
  "unidades_antes": 117,
  "decremento": 3,
  "unidades_despues": 114
}
```

CASO 2: No hay suficientes productos para decrementar

Resultado:

Response CODE: 409

```
{
  "detail": "Stock insuficiente de PRD001: hay 114,
se pidieron 1000"
}
```

CASO 3: El producto no existe

Response CODE: 409

```
{
  "detail": "El producto AAPRD001 no existe"
}
```

6. Conclusiones, limitaciones y posibles extensiones

El desarrollo de VetSalud permitió comprobar la utilidad de una arquitectura de persistencia polígota en la que ambos motores cumplen una función diferenciada. MongoDB resultó adecuado como fuente de verdad por su flexibilidad documental, su capacidad para ejecutar agregaciones y la posibilidad de realizar actualizaciones atómicas, como el decremento de stock. Redis, por su parte, permitió incorporar una capa de caché que reduce el tiempo de respuesta y evita repetir consultas costosas sobre la base principal.

La implementación de una API REST mediante FastAPI permitió exponer las consultas y servicios de forma uniforme, separando las responsabilidades entre las capas de rutas, servicios y acceso a datos. Esta organización mantiene la lógica de las consultas desacoplada de la estrategia de caché y facilita la incorporación de nuevas funcionalidades.

Sin embargo, la API implementada cubre principalmente las operaciones solicitadas por la consigna y todavía no constituye una interfaz completa para administrar todas las entidades del dominio. Como extensión podrían incorporarse operaciones POST, PUT y DELETE para pacientes, veterinarios, vacunaciones, cirugías y productos, además de funcionalidades para modificar consultas médicas, actualizar información del stock y consultar entidades individuales.

Cabe destacar que la incorporación de estos endpoints requeriría prestar especial atención a la estrategia de desnormalización. Actualmente, los documentos que representan eventos almacenan copias de algunos datos relacionados, como el nombre, la especialidad y la sucursal del veterinario, así como el identificador del propietario del paciente. Consecuentemente, al agregar nuevas operaciones de modificación sería necesario definir explícitamente qué campos desnormalizados representan una captura histórica y cuáles deben reflejar el estado actual. Ya que de no definir esto, podríamos empezar a tener inconsistencias de donde se encuentra el dato actualizado o pisar información valiosa. Por ejemplo, si un veterinario cambia de sucursal, las consultas anteriores podrían conservar la sucursal en la que fueron realizadas, ya que forman parte del historial clínico. En cambio, si algún dato desnormalizado debiera representar información vigente, su modificación tendría que propagarse a todos los documentos relacionados mediante actualizaciones masivas o procesos asíncronos. Sin estas reglas, distintas colecciones podrían presentar versiones contradictorias de una misma información.

Otra limitación de la implementación actual es que las modificaciones se confirman primero en MongoDB y luego se intenta invalidar la caché en Redis. Si Redis no está disponible en ese momento, la operación permanece correctamente almacenada en MongoDB, pero las claves relacionadas no pueden eliminarse. Mientras Redis continúa caído, las lecturas se resuelven directamente desde MongoDB y obtienen datos actualizados. Sin embargo, si Redis vuelve a estar disponible conservando las claves anteriores, podría servir temporalmente resultados desactualizados. Aunque el objetivo de el TTL es justamente funcionar como una red de seguridad frente a un escenario de este tipo, no garantiza una actualización inmediata. En consecuencia el sistema ofrece consistencia eventual y prioriza que una falla de la caché no provoque el rechazo de una escritura que ya fue confirmada en MongoDB, garantizando la durabilidad de los datos.